

高级 Agent 开发工程师 面试背诵手册

目标岗位：本智激活 | 高级 Agent 开发工程师

候选人：项思哲

35 道高频问题 | 60 ~ 90 秒口述答案 | 追问关键词 | 项目证据边界

生成日期：2026-06-19

目标岗位：本智激活 | 高级 Agent 开发工程师 候选人：项思哲 使用方式：先背每题的“核心句”，再用“口述版”扩展到 60~90 秒。追问只记关键词，不要逐字背诵。

回答总原则

1. 先说结论，再说架构、实现、权衡和结果。
2. 项目题优先使用“背景—职责—方案—难点—结果”结构。
3. 做过的内容明确说“我负责”；没有完整落地的内容说“我的设计会是”，不要包装成真实经历。
4. Agent 岗位重点突出四件事：状态可见、调用可控、异常可恢复、过程可追踪。
5. 遇到不会的细节，可以说明已掌握的相邻经验、风险认识和验证计划。

一、React、TypeScript 与前端架构

1. 从零搭建 Agent Web 客户端

核心句：我会按领域拆分对话、任务、工具调用和知识库，用事件协议连接 Agent 后端，把业务核心与 Web/Electron 容器隔离。

口述版：

我会把系统分成基础设施层、领域层和页面层。基础设施层封装 HTTP、WebSocket、鉴权、日志和本地存储；领域层拆成 conversation、task、tool-call、knowledge-base 等模块；页面层负责路由与组合。组件内部交互用本地状态，跨组件的任务状态放 Zustand 或 Redux，可分享和可回退的筛选条件放 URL，服务端数据通过查询缓存管理。Agent 返回的事件先进入统一事件适配器，再更新领域状态，避免组件直接理解后端协议。为了支持 Electron，我会把文件系统、系统通知等能力定义成统一接口，Web 端和 Electron preload 分别实现，React 业务层保持不变。

追问关键词：

- 本地状态：输入框、弹窗、临时选中。
- 全局状态：当前任务、连接状态、跨页面会话。
- URL：搜索词、筛选条件、当前知识库。
- 服务端缓存：会话列表、历史消息、知识文档。
- Electron 复用：端口适配器、能力接口、禁止业务层直接调用 Node API。

2. React 渲染与流式输出优化

核心句：流式输出的关键不是完全不渲染，而是控制更新频率、缩小更新范围，并用 Profiler 验证。

口述版：

React 更新大致经历触发更新、render/reconciliation 和 commit。render 阶段计算新树并比较差异，commit 阶段把变化写入 DOM 并执行相关副作用。Agent token 如果每到一个就更新整个消息数组，会造成高频 reconciliation。我会把正在生成的消息与历史消息分离，使用缓冲区聚合 token，每 16~50 毫秒批量刷新；列表项使用稳定 key，并将历史消息组件 memo 化。Markdown 解析也不能每个 token 全量执行，可以分块或在完成后做最终渲染。最后通过 React Profiler、Performance 面板和长任务数据比较优化前后的提交次数和耗时。

追问关键词：

- `React.memo`：适合 props 稳定、渲染成本较高的组件。
- `useMemo/useCallback`：用于稳定昂贵计算或引用，不要全项目机械添加。
- 高频 token：缓冲、节流、批量更新、局部状态。
- 证据：commit 次数、主线程耗时、INP、长任务。

3. Hooks 闭包与长连接

核心句：Hook 回调捕获的是创建时那次渲染的数据，长连接回调必须通过正确依赖、函数式更新或 ref 获取最新值。

口述版：

React 每次渲染都会创建新的函数和变量快照。WebSocket 监听器如果只在首次挂载时注册，回调可能一直引用旧状态，这就是 stale closure。纯状态累加可以使用函数式更新，例如 `setMessages(prev => [...prev, message])`；需要读取最新配置时，可以同步到 ref，再由稳定回调读取。若连接确实依赖用户或会话，则把依赖写完整并在 `cleanup` 中取消旧监听。我的原则是连接生命周期和业务状态更新分开，避免状态变化导致反复建连，也避免组件卸载后监听器仍然存在。

追问关键词：

- 依赖遗漏：旧值、逻辑不更新、难复现。
- `useRef`：保存最新值但不触发渲染。
- 函数式更新：基于前值计算新值。
- 防重复监听：稳定 handler、`cleanup`、连接单例、订阅计数。

4. TypeScript Agent 事件协议

核心句：我会用带 `type` 字段的可辨识联合描述事件，并在网络边界增加运行时校验。

口述版：

Agent 流式事件可以统一包含 `version`、`taskId`、`sequence`、`timestamp`、`type`、`payload`。事件类型使用可辨识联合，例如 `text.delta`、`tool.started`、`tool.completed`、`task.status`、`error` 和 `done`，前端通过 `switch` 穷尽处理。TypeScript 只能约束编译期，服务端数据进入系统时还要通过 `schema` 做运行时校验，非法事件进入错误通道。协议升级时保持新增字段向后兼容，重大变化提升 `version`，并为旧版本保留适配器。这样后端新增事件后，前端未处理的分支可以在编译期暴露。

追问关键词：

- 联合类型：每种事件有独立 `payload`。
- 穷尽检查：`never`。
- 运行时校验：网络数据不可信。
- 协议兼容：`version`、可选字段、`adapter`、契约测试。

5. 事件循环与流式 UI

核心句：微任务会在当前任务结束后、渲染前被清空，因此连续微任务可能推迟浏览器渲染。

口述版：

浏览器执行一个宏任务后会清空微任务队列，再进入样式计算、布局和绘制。Promise 回调属于微任务，如果流式解析不断产生新的微任务，浏览器可能长时间没有机会渲染。对于高频消息，我会先写入内存缓冲，再通过 `requestAnimationFrame` 或节流函数批量提交 UI；重计算可以切片，必要时放到 Web Worker。定位时使用 Performance 面板观察 long task、事件循环阻塞、布局和绘制成本，而不是只看接口速度。

追问关键词：

- 微任务：Promise、`queueMicrotask`。
- `requestAnimationFrame`：下一帧绘制前更新视觉状态。
- 长任务：通常超过 50ms。
- 优化：批处理、任务切片、Worker、减少布局抖动。

二、Electron 桌面端

注意：简历中没有完整 Electron 商业项目。回答时使用“我理解的架构和验证方案”，不要说成已大规模上线。

6. Electron 进程模型

核心句：主进程管理系统资源和窗口，渲染进程负责 React UI，preload 是受控能力桥梁。

口述版：

Electron 主进程运行在 Node.js 环境，负责应用生命周期、BrowserWindow、菜单、文件、系统通知和原生能力。渲染进程运行页面和 React，默认不应该直接拥有 Node 权限。preload 在隔离上下文中通过 contextBridge 暴露白名单 API。我的设计原则是主进程掌握高权限资源，渲染进程只提交明确请求，preload 负责最小化接口和参数转换。窗口关闭时需要区分隐藏、销毁和应用退出，并清理对应 IPC、定时器和子进程。

追问关键词：

- contextIsolation: true。
- nodeIntegration: false。
- 主进程：窗口、文件、系统、更新、子进程。
- preload：白名单 API，不暴露整个 ipcRenderer。

7. Electron IPC 设计

核心句：IPC 应当像内部 API 一样有白名单、类型、校验、超时、取消和统一错误码。

口述版：

请求响应场景使用 ipcRenderer.invoke 与 ipcMain.handle，持续事件使用订阅式 send/on。preload 只暴露具体能力，例如 readKnowledgeFile，而不是通用 invoke(channel, args)。主进程对来源、channel 和参数做校验，并限制可访问路径和操作类型。耗时任务返回 taskId，再通过事件推送进度，同时支持 AbortController 或取消 IPC。错误统一序列化为业务错误码，避免把堆栈和系统路径直接暴露给渲染进程。

追问关键词：

- invoke：一次请求一次响应。
- send/on：事件和持续进度。
- 安全：白名单、schema、路径沙箱、来源校验。
- 稳定性：taskId、超时、取消、错误归一化。

8. Electron 打包与升级

核心句：打包的难点集中在资源路径、原生依赖、签名、公证和升级兼容，必须通过多平台 CI 和灰度发布控制风险。

口述版：

开发和生产环境的资源路径不同，asar 内外资源、动态二进制和原生模块需要明确配置。原生依赖必须与 Electron 使用的 Node ABI 匹配。发布阶段，Windows 需要签名，macOS 需要签名和 notarization。自动升级要区分强制与可选更新，升级前保存任务状态，服务端和本地数据结构要向后兼容。版本发布采用灰度、校验安装包签名，并保留上一稳定版本和失败回滚机制。

追问关键词：

- `app.getPath` 管理用户数据。
- 原生模块 rebuild。
- `asar unpack`。
- 签名、公证、增量更新、灰度、回滚。

9. Electron 故障定位

核心句：先按主进程、渲染进程、IPC 和外部资源分层，再用日志、性能快照和崩溃报告定位。

口述版：

启动慢先拆分主进程初始化、窗口创建、页面加载和 Agent 服务启动耗时。白屏先检查渲染进程崩溃、资源地址、CSP、preload 加载和 IPC 异常。内存上涨要分别看主进程与渲染进程，重点检查未销毁窗口、事件监听器、WebSocket、定时器、缓存和子进程。生产环境应记录 app version、OS、窗口、taskId 和关键生命周期事件，并接入 crash reporter。问题修复后要补充相应的自动化测试和资源清理检查。

追问关键词：

- 主进程：启动日志、CPU、子进程、窗口引用。
- 渲染进程：DevTools、heap snapshot、Performance。
- 清理：IPC、socket、timer、worker、object URL。
- 线上证据：版本、OS、taskId、崩溃堆栈。

三、Agent、MCP、Skills 与 RAG

10. Agent、Workflow 与普通对话

核心句：普通对话是一次生成，Workflow 是预定义流程，Agent 会根据状态动态选择下一步行动。

口述版：

普通大模型对话主要是输入上下文后生成回答；Workflow 的步骤、分支和重试通常提前定义，适合稳定业务流程；Agent 则会根据目标、观察结果和工具返回动态决定下一步，灵活但不确定性更高。前端不能只显示最终文本，还要展示任务状态、当前步骤、工具调用、等待确认、重试、失败和完成。涉及写文件、执行命令或外部提交时，应在操作前让用户确认，并支持取消和恢复。

追问关键词：

- Workflow：稳定、可预测、易测试。
- Agent：动态规划、工具选择、不确定。
- UI 状态：queued/running/waiting/succeeded/failed/cancelled。
- 高风险操作：解释影响、用户确认、审计。

11. MCP 基本概念

核心句：MCP 用标准协议连接模型应用和外部能力，降低每个客户端重复集成工具与数据源的成本。

口述版：

MCP 中，Host 是承载模型和交互的应用，Client 负责与某个 MCP Server 建立会话，Server 暴露能力。Tool 是可执行操作，Resource 是可读取的上下文数据，Prompt 是可复用提示模板。它与 REST API 不冲突，REST 解决服务接口，MCP 在其上提供面向模型的能力发现、schema 和调用语义。Function Calling 主要描述模型如何产生工具参数，MCP 进一步标准化工具来自哪里、如何发现和通信。

追问关键词：

- Tool：有副作用或执行能力。
- Resource：读取上下文。
- Prompt：可复用交互模板。
- stdio：本地进程，简单隔离。
- HTTP：远程、多客户端、需要鉴权。

12. 多 MCP Server 治理

核心句：多 MCP 的核心不是“能连上”，而是最小权限、独立生命周期、故障隔离和全链路审计。

口述版：

每个 MCP Server 应独立配置、启动、健康检查和关闭，单个服务崩溃不能拖垮整个 Agent。工具按风险分级，只读工具可自动执行，写文件、执行命令和外部提交需要确认。UI 展示工具来源、参数摘要、执行状态、耗时和结果。调用记录关联 taskId、serverId、toolName 和 traceId，并对敏感参数脱敏。本地服务可运行在受限子进程中，设置超时、并发限制和允许访问的目录。

追问关键词：

- 权限：白名单、最小权限、风险分级。
- 隔离：独立进程、超时、熔断。
- UI：来源、参数、状态、结果、确认。
- 审计：谁、何时、调用什么、结果如何。

13. Skills 的边界

核心句：Tool 是原子能力，Prompt 是表达方式，Workflow 是固定流程，Skill 是完成某类任务的可复用操作知识包。

口述版：

我把 Skill 理解为围绕一个明确任务组织的说明、步骤、约束、工具使用方式、模板和资源。Tool 负责“能做什么”，Skill 负责“怎样可靠地完成一类任务”；Prompt 只是其中的语言指令；Workflow 更强调预定义执行图。一个好的 Skill 应声明触发条件、输入输出、允许工具、安全边界、失败处理和质量标准。测试时既要验证典型输入，也要覆盖缺失输入、工具失败、恶意内容和结果一致性。

追问关键词：

- 元数据：名称、描述、触发条件、输入输出。
- 上下文：按需加载，避免一次塞入全部资料。
- 测试：黄金样例、边界样例、工具失败、回归集。
- 评估：成功率、步骤合规、结果质量、成本。

14. RAG 全链路

核心句：RAG 的质量由解析、切片、召回、重排和生成共同决定，不能只看向量数据库。

口述版：

首先解析文档并保留标题、页码、章节和权限等元数据；然后按语义和结构切片，生成 Embedding 并建立向量与关键词索引。查询阶段可以做改写和扩展，使用 Dense 与 Sparse 混合召回，再通过 Rerank 选出最相关片段，经过上下文压缩后交给模型回答，并返回引用。失败可能发生在解析丢结构、切片断语义、召回缺失、重排错误、上下文过长或模型忽略证据。评估时必须把“召回正确”和“回答正确”分开。

追问关键词：

- 表格：保留表头、行列关系和来源。

- 代码：按符号或函数切片。
- 标题：作为父级上下文。
- 诊断：先看 top-k 是否包含答案，再看生成是否引用。

15. RAG 前端展示

核心句：RAG 界面要让用户知道答案来自哪里、证据是否充分，以及系统在哪一步不确定。

口述版：

答案中的关键结论应关联引用编号，点击后展示原文片段、文档名、章节和页码，并支持回到原文位置。检索结果可以显示相关度、关键词命中和更新时间，但不把内部分数包装成绝对可信度。资料冲突时并列展示来源并提示差异，证据不足时明确说“当前知识库未找到充分依据”。高级视图可以展示查询改写、召回、重排和生成耗时，普通用户默认只看答案和引用。

追问关键词：

- 可解释：引用、原文定位、更新时间。
- 冲突：并列证据，不替用户隐藏。
- 低置信：明确提示、建议补充问题。
- 性能：阶段耗时、首 token、总耗时。

四、WebSocket、Node.js 与工程交付

16. WebSocket Agent 流式协议

核心句：协议必须有 taskId、sequence、事件类型和版本，才能支持乱序处理、恢复和升级。

口述版：

我会统一消息结构：version、taskId、messageId、sequence、timestamp、type、payload。事件包括任务开始、文本增量、工具开始、工具结果、状态变化、错误和完成。客户端按 taskId 分流，按 sequence 去重和排序；收到 done 后关闭该任务订阅。用户取消时发送 command 事件，服务端返回 cancelling 和 cancelled。页面刷新后通过 taskId 查询快照，再从 lastSequence 补增量事件。新增事件保持向后兼容，破坏性变化提升协议版本。

追问关键词：

- 去重：messageId/sequence。
- 恢复：任务快照 + 增量补偿。
- 取消：命令确认，不能只在前端隐藏。
- 升级：version、能力协商、兼容 adapter。

17. 心跳与断线重连

核心句：心跳负责发现半开连接，重连使用指数退避加随机抖动，并通过任务快照恢复业务状态。

口述版：

客户端维护最近一次服务端消息或 pong 时间，超过阈值主动断开并重连。重连延迟采用指数退避并加入 jitter，避免大量客户端同时冲击服务器。浏览器离线时暂停重连，网络恢复后立即尝试；页面从休眠恢复时重新检查连接。每次连接都生成 connectionId，订阅使用 taskId 去重，旧连接回调失效。连接恢复后不能假设消息没丢，需要用 lastSequence 拉取缺失事件或重新获取任务快照。

追问关键词：

- 网络中断：online/offline 只是辅助信号。
- 服务端关闭：close code/reason。
- 浏览器休眠：时间差检测。
- 防重复：连接代次、订阅集合、幂等恢复。

18. Node.js 中间层稳定性

核心句：长任务必须从同步请求模型升级为任务模型，并具备超时、取消、限流、幂等和链路追踪。

口述版：

前端先创建任务获得 taskId，再通过 WebSocket 订阅事件。Node.js 中间层负责鉴权、协议转换、Agent 服务编排和事件转发。每层设置合理超时，只有幂等操作才自动重试；取消信号要传递到下游模型或工具。并发量通过用户级和系统级队列限制，流速过快时做背压或聚合。日志统一带 traceId、taskId、userId、toolCallId 和阶段耗时，方便从前端错误追到后端节点。

追问关键词：

- 幂等：idempotency key、任务唯一键。
- 背压：缓冲上限、批量发送、暂停读取。
- 限流：用户、模型、工具三级。
- 日志：traceId/taskId/sequence/latency/errorCode。

19. Agent 模块测试

核心句：测试要覆盖状态机和故障恢复，而不仅是最终成功页面。

口述版：

单元测试覆盖事件 reducer、协议解析、重连策略和状态转换；组件测试覆盖工具确认、错误提示和流式渲染；契约测试确保前后端事件 schema 一致；端到端测试覆盖创建任务、流式返回、取消和恢复。

WebSocket 使用可编程 mock server 模拟乱序、重复、断线、超时、错误事件和服务端重启。生产事故对应的事件序列要固化为回归用例。

追问关键词：

- 单元：纯函数、状态机。
- 组件：用户操作和可见状态。
- 契约：事件 schema、版本兼容。
- E2E：真实链路关键路径。

20. 代码评审与工程规范

核心句：规范应由自动化承担，评审重点放在设计、边界、风险和可维护性。

口述版：

格式、lint、类型检查、单测和构建放入 pre-commit 或 CI，避免评审者讨论机械问题。功能按垂直切片提交，每个 PR 说明背景、方案、风险和验证方式。高风险模块需要设计评审和至少一名领域负责人 review。对于分歧，先明确业务目标和约束，再用可验证的指标或小型 spike 比较方案。我的目标是让评审尽早发现问题，而不是在最后成为审批瓶颈。

追问关键词：

- CI 阻断：类型错误、测试失败、安全问题、构建失败。
- 小提交：每个提交可理解、可回退。

- 分歧：目标、约束、证据、实验、决策记录。
- 组长经验：任务拆分、需求评估、组件库和交付协调。

五、BI 智能问数 Agent 项目

21. BI 问数完整链路

核心句：我负责把自然语言问题转换为可执行、可恢复、可展示的数据查询流程。

口述版：

用户输入问题后，系统先做关键词扩展和意图理解，再从元数据知识库检索相关表、字段、指标口径和候选值。LangChain 封装字段理解、指标解析和 SQL 生成等 LLM 节点，LangGraph 负责节点编排、状态流转和 checkpoint。生成 SQL 后执行查询，将结果整理成表格或图表数据。后端通过 FastAPI 和 SSE 持续推送当前步骤，前端展示处理状态、文本结果和图表。我的职责覆盖 Agent 节点、流程编排、元数据检索、接口和前端交互。

追问关键词：

- LangChain：节点能力封装。
- LangGraph：流程、分支、状态、恢复。
- 前端：过程可视、结果图表化。
- 难点：业务口径歧义、字段映射、错误定位。

22. LangGraph 与 checkpoint

核心句：SQL Agent 不是一次模型调用，而是有状态、多阶段、可能失败和恢复的流程，因此适合状态图。

口述版：

我选择 LangGraph 是因为问数包含理解、检索、生成、校验、执行和结果解释，多节点之间存在条件分支。状态中保存标准化问题、候选元数据、生成 SQL、校验结果、执行摘要和错误信息；大结果集或敏感数据不直接塞入 checkpoint，只保存引用。checkpoint 让任务失败后从安全节点恢复，也支持后续加入人工确认。节点要尽量设计成幂等，数据库执行等有副作用步骤需要记录执行标识，避免恢复时重复执行。

追问关键词：

- 状态：必要、可序列化、可审计。
- 不保存：密钥、大结果集、无关上下文。
- 人工审批：SQL 生成后进入 interrupt/waiting。
- 恢复：checkpoint + 幂等节点。

23. MySQL、Qdrant、Elasticsearch 分工

核心句：MySQL 保存权威结构化元数据，Qdrant 做语义召回，Elasticsearch 做关键词和精确匹配。

口述版：

表结构、字段、指标口径和版本关系属于权威结构化数据，适合 MySQL。用户可能使用业务别名或自然语言表达，Qdrant 用向量检索召回语义相近内容。表名、字段名、编码值和专有词对精确匹配要求高，Elasticsearch 更合适。查询时融合向量与关键词候选，再按表关系、字段可用性和业务域重排。元数据更新以 MySQL 为源，通过版本或事件更新两个检索索引，索引记录 sourceVersion 以发现不一致。

追问关键词：

- 切片：表级、字段级、指标级、候选值级。
- 融合：加权、RRF、业务规则重排。
- 一致性：源版本、增量更新、失败重试、定期校验。
- 取舍：不能为了技术栈丰富而重复存储。

24. SSE 迁移到 WebSocket

核心句：可以复用任务状态和事件模型，主要改造传输层、双向命令和连接恢复机制。

口述版：

SSE 适合服务器单向推送，基于 HTTP，浏览器原生重连简单；WebSocket 是全双工，适合任务取消、人工确认和多任务订阅。迁移时我会保留现有任务状态机，把 SSE 消息统一成事件协议，再增加 WebSocket transport。前端领域层只消费事件，不关心传输方式。WebSocket 需要额外实现心跳、sequence、重连、补偿和订阅管理。协议通过 version 和可辨识事件保持兼容。

追问关键词：

- 可复用：taskId、状态机、事件类型、UI reducer。
- 新增：双向命令、心跳、乱序去重、恢复。
- 多任务：按 taskId 订阅和分流。
- 迁移：先双协议并行，再逐步切换。

25. SQL Agent 安全与评估

核心句：生产级 SQL Agent 必须把模型当作不可信输入，通过权限、静态校验、执行限制和评测集共同控制。

口述版：

我项目中已完成 SQL 生成与查询链路；如果按生产标准完善，我会使用只读账号和库表字段白名单，结合用户的数据权限生成可访问 schema。执行前进行 SQL 解析，只允许 SELECT，禁止多语句和危险函数，并通过 EXPLAIN 检查扫描量、超时和 LIMIT。准确率评估拆成元数据召回率、SQL 执行成功率、结果正确率和最终回答正确率。每次任务保留问题、召回内容、提示版本、SQL、执行摘要和节点状态，错误可以定位到具体阶段。

追问关键词：

- 权限：只读账号、白名单、行列权限。

- 校验：AST、SELECT only、LIMIT、EXPLAIN、timeout。
- 评测：标准问题集、标准 SQL/结果、业务专家审核。
- 诚实边界：安全治理是完善方向，不夸大现有落地范围。

六、BI 报表与前端工程化

26. BI 报表平台架构

核心句：报表平台的核心是稳定的配置协议，编辑器、渲染器、仪表盘和导出都围绕同一份 schema 工作。

口述版：

字段区负责提供维度和指标，编辑器将拖拽操作转换为报表 schema，schema 描述数据源、字段、聚合、筛选、排序、图表和样式。渲染器只消费 schema 和查询结果，仪表盘负责布局、联动和全局筛选，导出模块也基于同一配置生成结果。协议带 version，并通过迁移函数兼容历史报表。图表之间通过明确的联动事件交互，避免直接共享整个全局状态。

追问关键词：

- schema：数据、查询、展示、交互分层。
- 版本：version + migration。
- 联动：发布筛选条件，目标组件自行响应。
- 导出：相同查询口径与格式化逻辑。

27. Storybook 与管道模式

核心句：管道模式把数据转换、默认配置、主题、业务扩展和最终渲染拆成可组合步骤。

口述版：

不同项目对图表的差异主要来自数据结构、默认样式和业务配置。如果把所有分支写进组件，组件会迅速膨胀。我使用管道模式，让输入数据依次经过标准化、配置合并、主题处理、业务插件和最终 ECharts option 生成。每个步骤职责单一，可替换、可测试。Storybook 用来独立展示组件状态、边界数据和交互，通过 npm 包让多个项目复用。版本发布遵循语义化版本，破坏性变化提供迁移说明。

追问关键词：

- 管道价值：开放扩展、减少条件分支。
- 测试：每个 processor 单测，Storybook 场景测试。
- 发布：semver、changelog、兼容层。
- 风险：管道过细会增加理解成本。

28. VTable 适配器模式

核心句：适配器把业务字段协议转换成 VTable 配置，隔离第三方 API 与业务模型。

口述版：

业务层只描述字段类型、格式、权限、下钻和交互，不直接依赖 VTable 的列配置。适配器根据字段类型选择文本、数字、日期、枚举等渲染器，并生成列宽、冻结、排序和事件配置。这样替换或升级表格库时，变化集中在适配层。大数据量下避免把完整数据复制到多个状态，使用虚拟化、稳定列定义和局部更新；复杂单元格渲染器也要避免创建大量闭包和 DOM。

追问关键词：

- 隔离：第三方 API、事件模型、格式配置。
- 类型安全：字段类型到 renderer 的映射表。
- 性能：虚拟滚动、稳定引用、批量更新。
- 下钻：业务事件，不让表格组件直接操作路由。

29. 页面加载优化

核心句：CDN 优化传输，异步加载减少首包，骨架屏改善感知，多区域部署降低网络距离，最终都要用指标验证。

口述版：

图片 CDN 通过压缩、格式转换和就近分发降低资源耗时；组件异步加载将非首屏功能拆包；骨架屏减少等待时的空白感，但不能掩盖长期慢请求；多区域部署降低跨地域延迟。验证时观察 LCP、INP、CLS、首屏资源大小、接口延迟和错误率，并做优化前后对比。静态资源使用内容 hash 长缓存，HTML 短缓存，多区域发布需要同版本和可回滚。

追问关键词：

- LCP：主要内容出现速度。
- INP：交互响应。
- CLS：布局稳定。
- 骨架屏风险：与真实布局不一致、闪烁、掩盖慢接口。

七、实时系统、全栈能力与岗位匹配

30. WebSocket 高频训练数据

核心句：高频实时页面要把“接收频率”和“渲染频率”分开，并保证连接、订阅和缓存都能清理。

口述版：

智慧靶场中 WebSocket 持续接收训练和轨迹数据。我会按消息类型和时间戳进入内存缓冲，业务计算按顺序处理，UI 使用固定帧率批量刷新，避免每条消息触发 React 渲染。协议最好包含 sequence 和 timestamp，客户端可以发现重复和缺口。断线后重连并请求快照或缺失区间。组件卸载时关闭订阅、定时器、动画帧和大对象引用，长期运行时通过 heap snapshot 检查内存增长。

追问关键词：

- 乱序：sequence 排序、窗口缓冲。
- 积压：丢弃过期中间帧、保留最新状态。
- 可视化：批量 setOption、减少重排。
- 泄漏：listener、timer、WebSocket、Unity 实例。

31. 视频、轨迹与 Unity 同步

核心句：三条链路必须使用统一时间基准，通过时间戳和缓冲窗口对齐，而不是依赖消息到达顺序。

口述版：

RTSP 经 Node.js 转为浏览器可播放的 MPEG 流，轨迹和命中事件通过 WebSocket 到达，Unity WebGL 负责三维呈现。服务端为视频片段和事件使用统一时间戳，客户端维护短缓冲，根据播放时间选择对应轨迹和事件。网络抖动时允许小范围延迟换取同步，差距过大时跳到最新关键状态。白屏问题按转码进程、网络流、播放器和 WebGL 分层排查；卡顿则看解码、主线程、Unity 渲染和数据更新频率。

追问关键词：

- 时间基准：服务端统一时间。
- 抖动：jitter buffer。
- 校准：播放时间与事件时间差。
- 分层：源流、转码、传输、解码、渲染。

32. NestJS TCP 设备接入

核心句：TCP 是字节流，必须用连接缓冲区按协议长度拆帧，不能假设一次 data 就是一条完整报文。

口述版：

每个设备连接维护独立缓冲区，收到数据后追加，再根据协议头、长度字段和校验位循环判断是否有完整报文。数据不足就等待下一次，存在多帧则连续解析。解析后先保存原始报文，再生成结构化结果，方便审计和重放。连接管理记录设备标识、最后心跳、异常次数和当前状态。对于异常设备设置报文大小上限、解析错误阈值和断开策略，写入速度跟不上时使用队列或批量持久化处理背压。

追问关键词：

- 粘包：一次读到多帧。
- 半包：一次只读到部分帧。
- 400+ 连接：连接表、心跳、超时清理、限流。
- 原始报文：追责、重放、协议升级。

33. 接口从十几秒降到约 2 秒

核心句：先用分段耗时和执行计划确认瓶颈，再针对扫描量、索引和返回数据量优化，最后用监控证明没有回退。

口述版：

我先记录接口总耗时、应用逻辑耗时和 SQL 耗时，确认主要瓶颈在数据库。然后检查慢 SQL 和 EXPLAIN，关注访问类型、使用索引、扫描行数、临时表和排序。结合查询条件调整索引和 SQL，减少不必要字段、重复查询与大结果集，并检查分页方式。优化后使用相同数据集对比响应时间和结果一致性，观察 CPU、数据库连接和写入影响。最终关键接口从十几秒降低到约 2 秒，并持续通过慢查询和接口监控防止回退。

追问关键词：

- 证据：APM、日志分段、slow query、EXPLAIN。
- 索引：联合索引顺序、覆盖索引、选择性。
- 风险：索引增加写入成本。
- 验证：相同条件、并发测试、结果一致。

34. Electron 经验缺口

核心句：我不会把“了解 Electron”说成完整商业项目，但 React、Node.js、实时通信和多端经验可以让我快速完成迁移。

口述版：

我的简历里没有完整的 Electron 商业项目，这是需要诚实说明的差距。但岗位需要的 React、TypeScript、Node.js 中间层、WebSocket、实时任务状态和线上排障，我都有实际项目经验。Electron 新增的核心是进程模型、IPC 安全、打包签名和桌面生命周期，我已经能够说明其架构和风险。为了验证交付能力，我会先完成一个最小 Agent 桌面客户端：React 界面、preload 白名单、IPC、文件导入、WebSocket 流式任务、打包和自动更新验证，再进入业务开发。

追问关键词：

- 可迁移：React、Node、协议、状态、性能、排障。
- 必补：IPC 安全、签名、公证、更新、原生依赖。
- 两周计划：骨架、能力桥、流式任务、打包、故障演练。
- 语气：诚实、有准备、能验证。

35. 主导技术方案与团队交付

核心句：我会用 BI 报表组件体系作为案例，突出需求拆分、方案取舍、复用建设和交付结果。

口述版：

在 BI 报表平台中，多个项目都需要图表、复杂表格和报表展示。如果每个项目独立开发，会造成重复建设和行为不一致。我参与梳理共性需求，基于 Storybook 建设图表组件库，通过管道模式处理数据和配置扩展，并以 npm 包支持多项目复用；复杂表格则通过 VTable 适配层隔离业务字段和第三方 API。推进过程中我会先确定公共能力边界，再拆分组件、适配器、文档和接入任务，通过评审和示例降低使用成本。结果是核心能力可以跨项目复用，也让后续需求迭代更集中。

追问关键词：

- 备选：业务内复制、通用大组件、组件库 + adapter。
- 取舍：不过度抽象，先覆盖稳定共性。
- 协作：需求评估、任务拆分、代码评审、接入支持。
- 责任：问题先止损，再定位、复盘和补测试。

八、开场与反问

1 分钟自我介绍

面试官您好，我叫项思哲，有接近 7 年前端和全栈开发经验，主技术栈是 React、TypeScript、Node.js，也做过复杂中后台、BI 报表、实时可视化和设备接入系统。最近一年我重点参与 AI 应用开发，完成过 BI 智能问数 Agent，使用 LangChain 和 LangGraph 编排字段理解、元数据检索和 SQL 生成流程，并通过 FastAPI、SSE 和 React 展示任务状态与图表结果。我过去也做过 WebSocket 高频数据、Node.js 推流、NestJS 接口和线上性能优化。这个岗位需要把 Agent 能力做成稳定的 Web 和 Electron 产品，我的优势是既能理解 Agent 链路，也能负责前端交互、实时通信和中间层落地。Electron 是我需要进一步补强的部分，但相关的 React、Node.js、协议设计和故障排查经验具备较强迁移性。

为什么选择这个岗位

这个岗位不是单纯调用大模型，而是要把 MCP、Skills、RAG 和 Agent 状态真正做成用户可操作的 Web 与桌面产品，这与我从 BI 产品前端逐步进入 Agent 全栈开发的经历非常匹配。我希望继续强化 Agent 产品工程化，尤其是桌面端能力、工具调用治理和长任务稳定性。

可以反问面试官

1. 当前 Agent 产品主要服务哪些用户场景，Web 与 Electron 的职责如何划分？
2. Agent 后端采用自主 Agent 还是固定 Workflow 为主，前端事件协议是否已经稳定？
3. MCP Server 是本地运行还是远程托管，目前如何做工具权限和用户确认？
4. 团队当前最需要解决的是功能交付、稳定性、桌面端工程化，还是 RAG 效果？
5. 这个岗位入职前三个月最重要的交付目标是什么？

最后检查

- 不把 Electron 描述成已有完整商业项目。
- SQL Agent 安全治理用“生产化完善方案”表达。
- 讲项目时必须说清自己的职责，避免全用“我们”。
- 每个技术答案至少包含一个取舍，不只背定义。
- 数字只使用简历已有事实：400+ 设备、日均 1 万+ 报文、接口十几秒降至约 2 秒。